Generate, Validate, Repair: Reliable Visual Data Reporting with LLM Agents

Naga Datha Saikiran Battula* Vrushali Koli† Bayan Divaaniaazar‡ Aritra Dasgupta§

New Jersey Institute of Technology

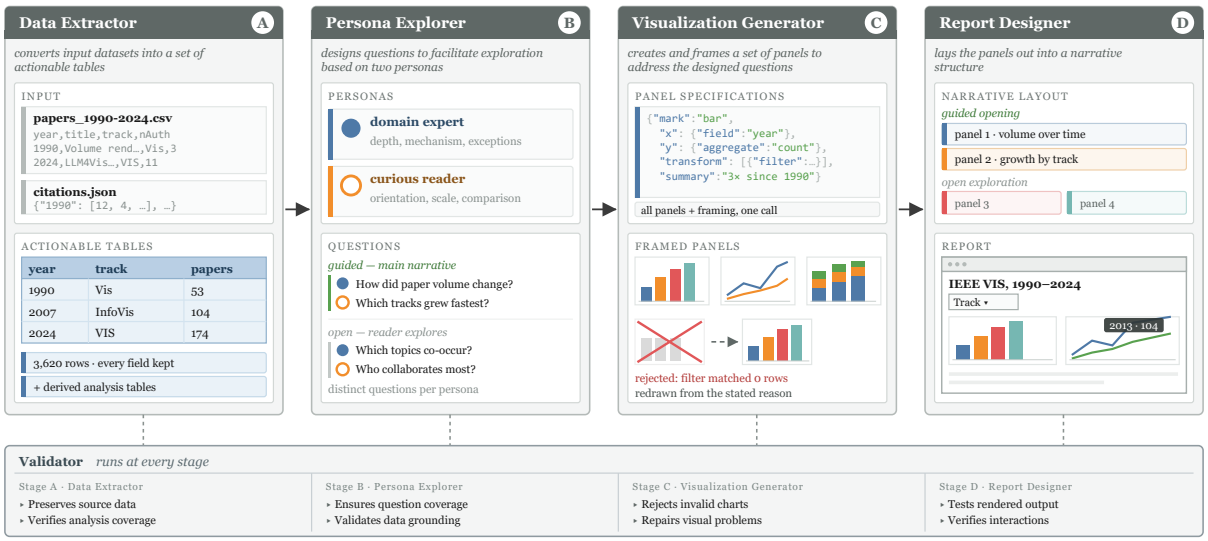


Figure 1: **A four-stage multi-agent pipeline for visual report generation.** The pipeline converts an input dataset into actionable tables (A), generates questions from a domain-expert and a curious-reader persona (B), authors and frames visualization panels in a single call (C), and lays them out as a guided narrative that opens into free exploration (D). A validator runs at every stage. At each stage, a validator rejects invalid outputs (e.g., zero-row panels) and sends repair instructions to the model, while maintaining a valid fallback.

Abstract

AI agents can now analyze, synthesize, and visualize complex datasets, yet turning their outputs into a coherent narrative aligned with a reader's expectations remains an open challenge. A single generation pass produces charts that misread the data, claims the visuals do not support, and narratives that lose coherence over a long document. We present a multi-agent system for automated visual data reporting in which every generative stage is paired with a deterministic validator that can reject its output, and failures are repaired in the same context or disclosed rather than shipped. The system organizes the validated panels into a guided narrative and renders it as an interactive report[1], letting readers follow the analytical thread and then explore the underlying data themselves.

Index Terms: Multi-agent systems, large language models, visualization generation, human-computer interaction

*e-mail: nb547@njit.edu
†e-mail: vk435@njit.edu
‡e-mail: bd347@njit.edu
§e-mail: aritra.dasgupta@njit.edu

[1]An example report is available at https://visagent.org/api/arena/v1/jobs/b420d5eb-d901-43bb-ad76-d7ef9e39a523/preview/index.html.

1 Introduction

Large Language Models (LLMs) have become ubiquitous in data analysis and visualization pipelines [10, 4]. Recent work in visual analytics leverages LLMs to produce data insights to help interpret visualizations [11, 5]. However, studies show that LLMs struggle with misalignment between the generated charts and data, narrative coherence, and textual integration in long-form reports [3, 8]. Automated visual reporting cannot be trusted if any stage can silently fail. We argue that reliable reporting requires pairing generation with verification. Each failure mode is specific and checkable, so each generative stage can be paired with a deterministic validator that rejects a violating output. We then repair a rejected output in the same context, or disclose it.

Unlike STORM [7], which drives multi-role text generation through question-answering, our system pairs generation with per-stage validation and organizes the results into a question-driven guided narrative, rendered as an interactive report that readers follow and then explore themselves. Reliability is a property of the report composition. The system parses the user's tasks once and threads them through every stage so each validates its output against the same asks, and no stage passes a result to the next stage that it cannot use. The system consists of four stages as shown in Figure 1:

Data Extractor. Converts input datasets and task briefs into a set of actionable tables

Persona Explorer. Designs questions to facilitate exploration based on two personas, a domain expert and a curious reader.

Visualization Generator. Creates and frames a set of panels to

address the designed questions.

Report Designer. Structures the panels into a martini-glass narrative structure [6] driving a guided analysis then exploration.

2 System Design and Implementation

The system organizes questions generated by two personas into a guided narrative and renders an interactive report. It accepts an input dataset in any format and an accompanying task brief, which can be detailed or state default report-creation criteria. Questions from the two personas provide the structure for the report's narrative. They connect each visualization to a specific information need and create a progression from a guided analytical thread to open exploration.

2.1 Design Principles

These three high-level principles established guidelines that were integrated across each stage.

Deterministic Validations. Every rule stated in the prompt has a Python counterpart that can reject a violation. If an instruction is enforced only through the prompt, we treat it as guidance. The model is asked to follow it, but the system cannot guarantee compliance.

Repair Over Blind Regeneration. When a single generation returns problems, we prefer to repair the artifact in the same conversation, letting the model edit visible work rather than re-derive blindly.

Pre-Render Specifications. Panels and framing are authored in one call so the narrative is a single thread, then each panel is validated alone. A panel that fails is regenerated, and one that still fails is dropped and disclosed rather than shipped as an empty frame.

2.2 Our Four-Stage Pipeline

Our multi-agent system consists of four stages, with the validator running at every stage. When the validator identifies an issue, the model receives the generated output and corresponding validation feedback for correction. We call this process repair. Since repeated corrections may not converge, we cap the repair process at two to three rounds per stage to avoid looping.

Data Extractor. The pipeline begins with understanding the data and transforming that dataset into an actionable set of metadata, relations, and statistical measures. First, we write a deterministic baseline that preserves every record and field, including four or five default insights computed over the full dataset. Then, the LLM is asked to improve upon this analytical exploration and generate a Python script to perform the extraction. If task briefs are complex, we split extraction into independent calls to ensure coverage. Validation is deterministic and ensures the extracted code executes safely by checking syntax parsing, the structural and semantic characteristics of the extracted tables, execution status, and restricted dependency use. On failure, extraction retries up to three times before falling back to deterministic validation to guarantee valid data.

Persona Explorer. The validated outputs from the *Data Extractor* are used to generate between 8-10 questions from the two personas. Drawing on the role of questions in data comprehension, we provide a few-shot prompt with positive and negative examples steering questions towards supporting diverse exploration [2, 9]. The two personas represent a domain expert and a curious reader with differing levels of expertise, analytical depth, and framing preferences. The task brief is split round-robin between both personas so generation remains anchored to it. Each question is tagged with the visualization task elicited from Multi-Level Typology [1], question complexity (low, medium, high), and narrative placement based on the martini-glass structure (stem, bowl) [6], which provide cues during the visualization generation. Validation ensures questions cover the brief and are grounded in extracted data.

Visualization Generator. Each question is transformed into a card layout. The model first fixes linking attributes shared across panels and a framing hook from the brief, then authors all panels and framing in one call so the narrative is a single thread. For each question, the model ranks up to three candidate sketches and selects one based on a decisive answer (does the summary name the leader/period and its value, on a measure that actually varies), answer legibility, interaction value, then simplicity. Here, validation happens at two points. We first check the generated specifications against the underlying data, including the transformations, filters, and encodings. However, passing these checks does not necessarily mean the visualization will render properly. Additionally, we check after rendering whether charts have consistent color encodings and visible data points. If a visualization fails, we flag it for repair. If the visualization still fails, we exclude the question panel.

Report Designer. The validated *Question Panels* are laid into a martini-glass narrative and rendered as a self-contained interactive web page. We ask the *Report Designer* to place each panel and textual framing into a report structure template. Here, the validation occurs on the rendered report, which checks layout, rendering, and interaction behavior. Detected problems, along with their focused strategy for locating the relevant portion of the report source, will be tagged for repair.

Through a combination of validation and a valid fallback at every stage, the four components produce a report whose faithfulness is a property of the pipeline as a whole, not of any component alone.

3 Discussion

Reliable reporting requires balancing two goals: narrative coherence and individual panel validation. Although panels are generated using validated data and design guidelines, an individual visualization may still contain an incorrect data mapping, an unclear answer, or a design that violates the report requirements. Panel-level validation helps identify these problems and allows the system to repair only the affected visualization. However, changing one panel may cause cascading effects downstream (e.g., outdated framing). Local repair should therefore preserve unaffected panels, followed by a final check that the questions, answers, visualizations, and framing remain aligned in the assembled report.

Our pipeline addresses both goals by authoring panels and framing in a single thread with shared linking attributes, then validating and repairing panels in isolation. A final check over the assembled report confirms that questions, answers, visualizations, and framing remain aligned. This end-to-end faithfulness is a property of the whole pipeline. The generator can reject a panel whose filter matched zero rows only because the extractor preserved the underlying rows to check against, and the report stays coherent only because authoring and validation span every panel at once. A limitation of this approach is that our guarantees are bounded by their validators. A rule enforces reliability only where it has a deterministic counterpart.

4 Conclusion

We presented a four-stage multi-agent pipeline, a *Data Extractor*, *Persona Explorer*, *Visualization Generator*, and *Report Designer*, that turns a dataset and a task description into an interactive report framed as a guided narrative. Pairing every generative stage with a deterministic validator makes each failure mode specific and checkable: a chart whose filter matches no rows, an answer the panel does not encode, or a control that changes nothing is rejected and repaired in the same context, or disclosed, rather than shipped. Because each stage retains a validated result to fall back to, no single stage can silently fail the report.

REFERENCES

[1] M. Brehmer and T. Munzner. A multi-level typology of abstract visualization tasks. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2376–2385, 2013. 2

[2] C. Chin and D. E. Brown. Student-generated questions: A meaningful aspect of learning in science. *International Journal of Science Education*, 24(5):521–549, 2002. 2

[3] M. S. Islam, M. T. R. Laskar, M. R. Parvez, E. Hoque, and S. Joty. DataNarrative: Automated data-driven storytelling with visualizations and texts. In Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, eds., *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 19253–19286. Association for Computational Linguistics, Miami, Florida, USA, Nov. 2024. doi: 10.18653/v1/2024.emnlp-main.1073 1

[4] W. Lan, Z. Tang, M. Liu, Q. Chen, W. Peng, Y.-p. P. Chen, and Y. Pan. The large language models on biomedical data analysis: a survey. *IEEE Journal of Biomedical and Health Informatics*, 29(6):4486–4497, 2025. 1

[5] A. Masry, J. Q. Tan, S. Joty, E. Hoque, et al. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, pp. 2263–2279, 2022. 1

[6] E. Segel and J. Heer. Narrative visualization: Telling stories with data. *IEEE Transactions on Visualization and Computer Graphics*, 16(6):1139–1148, 2010. 2

[7] Y. Shao, Y. Jiang, T. Kanell, P. Xu, O. Khattab, and M. Lam. Assisting in writing wikipedia-like articles from scratch with large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 6252–6278, 2024. 1

[8] N. Sultanum and A. Srinivasan. Datatales: Investigating the use of large language models for authoring data-driven articles. In *2023 IEEE Visualization and Visual Analytics (VIS)*, pp. 231–235. IEEE, 2023. 1

[9] A. A. Tawfik, A. Graesser, J. Gatewood, and J. Gishbaugher. Role of questions in inquiry-based instruction: Towards a design taxonomy for question-asking and implications for design. *Educational Technology Research and Development*, 68(2):653–678, 2020. 2

[10] X. Zhang and M. Wang. Large language models for financial knowledge extraction analytical insights and corporate planning support. *Math. Model. Algorithm Appl*, 6:44–56, 2025. 1

[11] Y. Zhao, Y. Zhang, Y. Zhang, X. Zhao, J. Wang, Z. Shao, C. Turkay, and S. Chen. Leva: Using large language models to enhance visual analytics. *IEEE Transactions on Visualization and Computer Graphics*, 31(3):1830–1847, 2024. 1